

Java Web 后端开发基础

从 CampusHub API 到可维护、可验证的服务

HTTP · Spring Boot · MyBatis · 安全与测试

本讲学习目标

完成本讲后，你应该能够：

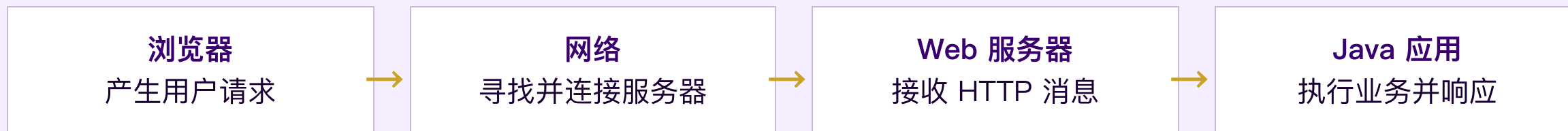
1. 解释 Web 请求在网络协议栈中的位置和传输过程；
2. 读懂 HTTP 请求、响应和常用状态码；
3. 为 CampusHub 设计清晰的 REST API 契约；
4. 使用 Spring Boot 实现 Controller、Service、Mapper 分层；
5. 使用 MyBatis 安全地读写关系数据库；
6. 处理参数校验、异常、事务和权限；
7. 在 AI 帮助下开发后端，并用测试验证结果。

目标不是记住所有注解，而是建立“请求如何穿过系统”的后端心智模型。

PART 1

从网络协议栈到 HTTP 请求

浏览器和后端如何跨网络通信？



用户点击“发布”

- 浏览器构造 HTTP 请求
- 网络协议逐层传输数据
- Tomcat 接收并解析 HTTP
- Spring MVC 调用对应 Controller

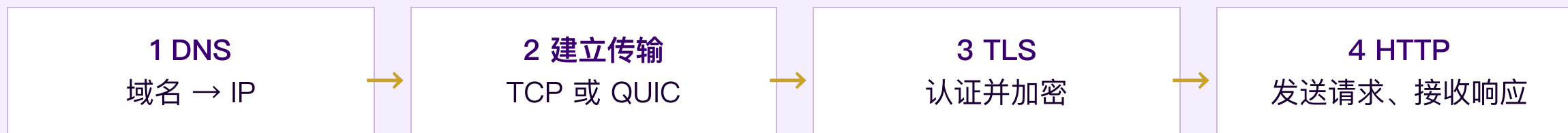
Java 方法并不是直接被浏览器调用；两者通过网络和 HTTP 消息协作。

TCP/IP 协议栈：每层解决一个问题

应用层	HTTP · DNS	表达请求语义、解析域名
↓ 调用 ↑ 返回		
传输层	TCP · UDP · QUIC	进程通信、可靠性与拥塞控制
↓ 调用 ↑ 返回		
网络层	IP	跨网络寻址与路由
↓ 调用 ↑ 返回		
链路层	Ethernet · Wi-Fi	在相邻设备之间传输帧

- 上层使用下层提供的能力，下层不理解上层业务；
- HTTP 是应用层协议，端口把网络数据交给具体服务器进程；
- 排查故障时要先判断问题属于域名、连接、加密还是应用响应。

一次 HTTPS 访问经历什么？



```
campushub.example.edu
  ↓ DNS 找到服务器地址
203.0.113.10:443
  ↓ 建立连接并协商加密
POST /api/demands HTTP/1.1
  ↓ 服务器处理后返回
HTTP/1.1 201 Created
```

HTTPS = HTTP + TLS 保护。 TLS 提供机密性、完整性和服务器身份校验，但不替代应用层权限检查。

数据如何逐层封装与还原？

发送端：逐层封装

HTTP Message

↓ TCP Segment / QUIC Packet

↓ IP Packet

↓ Ethernet / Wi-Fi Frame

接收端：逐层解封装

Frame 校验并取出 IP Packet

↓ 根据端口交给传输层连接

↓ 重组应用层字节

↓ Tomcat 解析 HTTP Message

后端开发主要处理 HTTP 语义，但连接超时、丢包、代理和 TLS 错误需要沿协议栈定位。

HTTP 的核心：请求—响应协议

客户端发起

浏览器或 App 主动发送请求

服务器响应

返回状态、头部和可选消息体

默认无状态

每个请求应携带处理所需信息

Request = Method + Target + Headers + Body?

Response = Status + Headers + Body?

- HTTP 定义消息语义，不规定 Java 方法如何实现业务；
- Cookie、Session 或 Token 用于在多次请求之间关联用户状态；
- HTTP/1.1 复用连接，HTTP/2 支持多路复用，HTTP/3 基于 QUIC。

今天要完成的后端能力

CampusHub · 发布需求

```
POST /api/demands
Content-Type: application/json

{"title":"帮忙取快递","category":"EXPRESS","location":"南园驿站"}
```

```
HTTP/1.1 201 Created
Location: /api/demands/101

{"id":101,"status":"OPEN","title":"帮忙取快递"}
```

从一个请求出发，理解：契约 → 路由 → 业务规则 → 数据库 → 响应。

后端到底负责什么？

接收请求

解析路径、参数、身份与请求体

执行规则

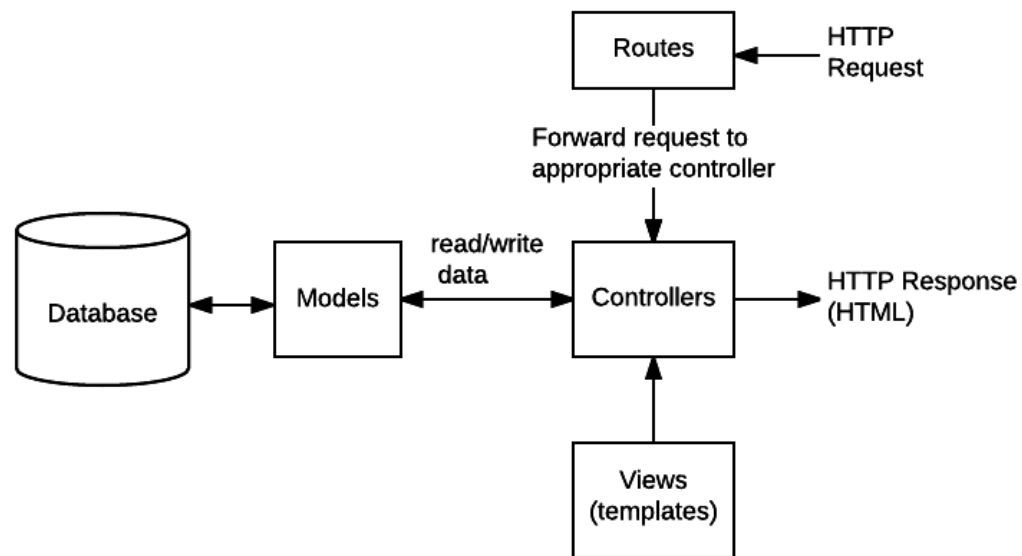
校验状态、权限与业务约束

管理数据

事务性地查询和持久化数据

前端反馈可以被绕过；后端必须保证任何调用方式下，业务规则仍然成立。

一次请求如何到达 Java 方法?



来源：MDN Web Docs, Express Routes and Controllers

路由根据 **HTTP 方法 + URL 路径** 选择处理函数；Controller 不应该直接承担全部业务和数据库工作。

HTTP 请求：后端需要读懂什么？

```
POST /api/demands?notify=true HTTP/1.1
Host: campushub.example.edu
Authorization: Bearer eyJ...
Content-Type: application/json

{"title":"帮忙取快递","category":"EXPRESS"}
```

部分	后端用途
Method + Path	匹配路由与操作
Query	筛选、分页或可选参数
Headers	身份、内容类型、追踪信息
Body	创建或更新的数据

URL：请求地址也有结构

https://campushub.example.edu:443/api/demands/101?notify=true#reviews

「协议」 「主机名」 「端口」 「路径」 「查询参数」 「片段」

部分	是否发送给后端	作用
协议、主机、端口	用于建立连接	找到目标服务
路径	是	定位资源并参与路由匹配
Query	是	筛选、分页和可选控制参数
Fragment #...	否	仅由浏览器处理页面内部位置

路径变量表示“哪个资源”， 查询参数表示“如何筛选或展示资源”。

URI、URL 与相对地址

URI：标识一个资源的通用概念
└ URL：既标识资源，也说明如何定位和访问它

写法	示例	浏览器如何解释
绝对 URL	<code>https://api.example.edu/demands/101</code>	包含完整协议、主机和路径
根相对 URL	<code>/api/demands/101</code>	从当前 Origin 的根路径开始
路径相对 URL	<code>../demands/101</code>	相对当前文档目录解析
Query 引用	<code>?page=2</code>	保留当前路径，替换查询部分
Fragment 引用	<code>#reviews</code>	只改变浏览器中的页面位置或前端状态

```
new URL('../demands/101', 'https://campus.edu/app/orders/7')  
// https://campus.edu/app/demands/101
```

不要用字符串猜测相对路径；浏览器、前端路由和反向代理都可能改变基准地址。

URL 编码：地址不是任意字符串

用户输入：学习辅导 & Java

正确 Query: ?keyword=%E5%AD%A6%E4%B9%A0%E8%BE%85%E5%AF%BC%20%26%20Java

```
const url = new URL('/api/demands', location.origin);  
url.searchParams.set('keyword', '学习辅导 & Java');  
url.searchParams.append('category', 'TUTORING');  
fetch(url);
```

- 空格、中文、&、=、# 等字符在 URL 中具有特殊含义，需要百分号编码；
- 使用 URL、URLSearchParams 或框架 API，不要手工拼接用户输入；
- 同名查询参数可以重复：?category=EXPRESS&category=TUTORING；
- + 在表单编码中常表示空格，但在路径中不是通用的空格写法；
- 后端应只解码到框架约定的层次，防止重复解码造成路径绕过。

Origin: 协议、主机和端口的组合

```
URL      = https://campushub.example.edu:443/api/demands?status=OPEN
Origin   = https://campushub.example.edu:443
Path     = /api/demands
Query    = status=OPEN
```

地址对比	是否同源	原因
https://campus.edu/a 与 https://campus.edu/b	是	协议、主机、端口相同
https://campus.edu 与 http://campus.edu	否	协议不同
https://api.campus.edu 与 https://campus.edu	否	主机不同
https://campus.edu:443 与 https://campus.edu:8443	否	端口不同

- 浏览器的同源策略限制页面读取跨 Origin 响应；
- CORS 是服务器明确允许指定 Origin 读取响应的机制；
- CORS 不是身份认证，非浏览器客户端也不受浏览器同源策略保护。

JSON：前后端交换的是数据结构

```
{  
  "title": "帮忙取快递",  
  "reward": 5.0,  
  "urgent": true,  
  "tags": ["快递", "南园"],  
  "location": { "building": "驿站", "floor": null }  
}
```

JSON 支持 `string`、`number`、`boolean`、`null`、对象和数组；请求头应声明 `Content-Type: application/json`。

反序列化

请求 JSON → Request DTO

序列化

Response DTO → 响应 JSON

字段不存在、字段为 `null`、空字符串和空数组具有不同语义，必须在 API 契约中明确。

HTTP 响应：不仅是返回 JSON

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
```

```
{
  "code": "INVALID_DEMAND",
  "message": "需求标题不能为空",
  "field": "title"
}
```

状态码	典型语义
200 OK	查询或更新成功
201 Created	创建成功
400 Bad Request	请求格式或参数错误
401 / 403	未登录 / 无权限
404 Not Found	资源不存在
409 Conflict	当前状态不允许操作
500	未预期的服务端错误

REST 来自 Fielding 的博士论文

Roy Thomas Fielding 在 2000 年博士论文《Architectural Styles and the Design of Network-based Software Architectures》第 5 章提出 REST。

REST = Representational State Transfer
= 表述性状态转移

REST 是什么？

面向分布式超媒体系统的一种**架构风格**，由一组相互配合的约束定义。

REST 不是什么？

不是编程框架、传输协议、JSON 格式，也不只是“URL 使用名词”。

论文关注组件的角色、交互约束及数据元素，而忽略具体实现和协议语法，以获得可伸缩性、通用接口和组件独立演化等性质。

来源：[Fielding Dissertation, Chapter 5: Representational State Transfer](#)

REST 的六项架构约束

约束	核心含义	带来的主要性质
Client—Server	UI 与数据/服务职责分离	独立演化、可移植性
Stateless	每个请求包含理解它所需的信息	可见性、可靠性、可伸缩性
Cache	响应明确是否可缓存	减少交互、改善性能
Uniform Interface	组件通过统一接口交互	简化架构、降低耦合
Layered System	客户端只需知道相邻层	代理、网关、负载均衡
Code—on—Demand	可下载脚本扩展客户端	可扩展性； 唯一可选约束

一个 API 使用 HTTP 和 JSON，不等于它自动满足 REST；需要看整体交互是否遵守这些约束。

REST 的核心：统一接口四项约束

1 资源可标识

每个资源通过 URI 稳定标识，例如 `/demands/101`。

2 通过表述操作资源

客户端传输 JSON 等表述，服务器据此改变资源状态。

3 消息自描述

方法、状态码、媒体类型和缓存头足以解释消息。

4 超媒体驱动状态

响应提供可继续执行的链接或动作，即 HATEOAS。

```
{
  "id": 101, "status": "OPEN",
  "links": [
    {"rel": "self", "href": "/api/demands/101"},
    {"rel": "acceptance", "href": "/api/demands/101/acceptance"}
  ]
}
```

来源：Fielding Dissertation §5.1.5 Uniform Interface 与 §5.2

课程中的 REST API 如何定义？

REST API：将业务对象和概念建模为资源，用 URI 标识资源，通过 HTTP 的统一语义交换资源表述，并保持请求无状态、响应自描述且可按语义缓存的 Web API。

设计问题	REST API 的回答
操作谁？	URI 标识资源或资源集合
做什么？	使用 GET 、 POST 、 PUT 、 PATCH 、 DELETE 的既有语义
传什么？	JSON 等资源表述，并声明 Content-Type
结果如何？	状态码、响应头和响应体共同表达
下一步是什么？	文档、链接或动作关系引导客户端

工程中常说的“RESTful API”通常只实现 REST 的一部分。应说明遵循了哪些约束，而不是简单贴标签。

REST：围绕资源设计 API

用户意图	方法	路径
浏览需求	GET	/api/demands?category=EXPRESS
查看需求	GET	/api/demands/{id}
发布需求	POST	/api/demands
修改需求	PATCH	/api/demands/{id}
接单	POST	/api/demands/{id}/acceptance
取消需求	POST	/api/demands/{id}/cancellation

URL 表达资源，HTTP 方法表达操作；复杂业务动作可以建模为子资源或明确动作端点。

CampusHub: REST API 正反示例

不推荐	问题	改进后的资源设计
GET /getDemand?id=101	路径表达动作, 参数另找资源	GET /api/demands/101
POST /createDemand	重复表达 HTTP 方法语义	POST /api/demands
POST /updateDemandStatus	动作和状态缺少资源边界	PATCH /api/demands/101
GET /deleteDemand/101	GET 产生副作用, 不安全	DELETE /api/demands/101
POST /acceptDemand?id=101	业务动作难以形成独立资源	POST /api/demands/101/acceptance

POST /api/demands/101/acceptance HTTP/1.1

Authorization: Bearer ...

HTTP/1.1 201 Created

Location: /api/acceptances/9001

接单可建模为创建“接单关系”资源; **Location** 指向新资源, 重复或冲突由明确状态码表达。

API 契约必须回答六个问题

输入

方法与路径是什么？

需要哪些参数和身份？

字段类型与约束是什么？

输出

成功返回什么数据？

失败使用什么状态码？

并发或重复请求如何处理？

先写契约和验收示例，再生成 Controller 代码。

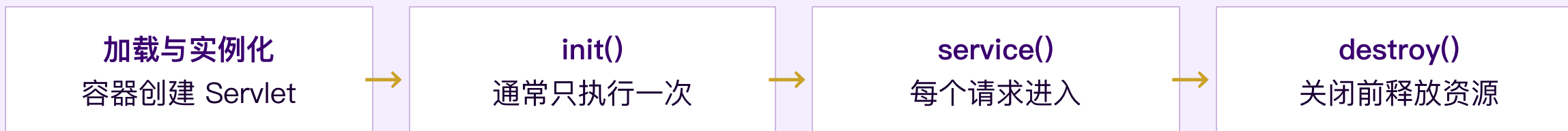
Tomcat 与 Servlet：理解一层即可

HTTP 请求 → Tomcat / Servlet 容器 → Spring MVC → Controller

- Tomcat 监听端口、解析 HTTP、管理线程与 Servlet 生命周期；
- `DispatcherServlet` 是 Spring MVC 的统一入口；
- Spring Boot 内嵌 Tomcat，应用可直接以 JAR 启动；
- `@RestController`、`@GetMapping` 等注解隐藏了 Servlet 样板代码。

不必从零手写 Servlet，但必须知道并发请求可能同时进入同一个 Controller 对象。

Servlet 生命周期与并发模型



```
public class CounterServlet extends HttpServlet {  
    private int count = 0; // 多个请求共享, 存在竞态条件  
  
    protected void doGet(HttpServletRequest req,  
                          HttpServletResponse resp) {  
        count++;  
    }  
}
```

- 容器通常使用多个线程并发处理请求，而不是为每个请求创建一个 Controller；
- Servlet、Controller、Service 默认应设计为**无状态组件**；
- 请求数据放在局部变量、DTO 或请求作用域中，不要放在可变实例字段中。

Spring MVC 的 `DispatcherServlet` 统一接收请求，再分派给 Controller 方法。

PART 2

Spring Boot 与分层设计

Spring Boot 解决了什么?

自动配置

根据依赖和配置装配常用组件

Starter

按场景组织兼容的依赖集合

内嵌服务器

应用可独立启动和部署

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```

Spring Boot 减少配置工作，但不会替你设计业务边界、数据模型和异常语义。

从 Initializr 到可运行项目

在 start.spring.io 选择 Java 17+、Maven，并按持久化方案加入依赖：

```
Spring Web Validation MyBatis Framework 或 Spring Data JPA
MySQL Driver Spring Security Spring Boot Test
```

```
src/main/java/edu/nju/campusHub/
├── CampusHubApplication.java
├── demand/
│   ├── DemandController.java
│   ├── DemandService.java
│   ├── DemandMapper.java      # 或 DemandRepository.java
│   └── Demand.java
└── common/                    # 异常、认证、共享配置
src/main/resources/
├── application.yml
└── mapper/DemandMapper.xml    # 使用 MyBatis XML 时
```

启动类放在根包，使组件扫描覆盖所有业务模块；中型项目优先按业务功能组织，再在模块内部保持分层。

常用注解放在正确边界

注解	位置	作用
@SpringBootApplication	启动类	自动配置、组件扫描和应用入口
@RestController	Controller 类	返回值写入 HTTP 响应体
@RequestMapping	类/方法	声明公共路径或请求映射
@GetMapping / @PostMapping	Controller 方法	按 HTTP 方法匹配路由
@RequestBody	方法参数	JSON 反序列化为 DTO
@PathVariable / @RequestParam	方法参数	读取路径变量 / 查询参数
@Valid	方法参数	触发 DTO Bean Validation
@Service / @Transactional	Service	标识业务组件 / 事务边界

注解只能声明框架行为，不能代替清晰的业务规则和测试。

最小可运行的 Controller

```
@RestController
@RequestMapping("/api/demands")
public class DemandController {
    private final DemandService demandService;

    public DemandController(DemandService demandService) {
        this.demandService = demandService;
    }

    @GetMapping("/{id}")
    public DemandResponse get(@PathVariable Long id) {
        return demandService.get(id);
    }
}
```

构造器注入让依赖显式、便于测试，也避免隐藏的可变依赖。

Controller 只负责 Web 边界

```
@PostMapping
public ResponseEntity<DemandResponse> create(
    @Valid @RequestBody CreateDemandRequest request,
    @AuthenticationPrincipal UserPrincipal user) {
    DemandResponse result = demandService.create(user.id(), request);
    URI location = URI.create("/api/demands/" + result.id());
    return ResponseEntity.created(location).body(result);
}
```

Controller 应负责：

- HTTP 参数绑定与格式校验；
- 调用一个清晰的应用服务；
- 将结果转换为 HTTP 响应。

不要在 Controller 中拼 SQL、开启事务或堆叠复杂业务判断。

DTO: 不要把数据库实体直接暴露给前端

```
public record CreateDemandRequest(  
    @NotBlank @Size(max = 80) String title,  
    @NotNull DemandCategory category,  
    @NotBlank String location  
) {}  
  
public record DemandResponse(  
    Long id, String title, String category, String status  
) {}
```

数据对象	主要职责
Request DTO	定义并校验 API 输入
Response DTO	稳定控制 API 输出
Entity / PO	映射持久化数据

Service 承载业务规则

```
@Service
public class DemandService {
    private final DemandMapper demandMapper;

    @Transactional
    public DemandResponse accept(Long demandId, Long providerId) {
        Demand demand = requireDemand(demandId);
        if (!demand.isOpen()) throw new DemandConflictException();
        if (demand.ownerId().equals(providerId)) throw new SelfAcceptException();

        demandMapper.markAccepted(demandId, providerId);
        return DemandResponse.from(demand);
    }
}
```

业务规则应该能脱离 HTTP 被单独测试。

分层不是“多建几个包”

Web / API 层	@RestController · DemandController	接收 HTTP、校验 DTO、返回状态码
↓ 调用 ↑ 返回		
业务层	@Service · DemandService	业务规则、状态转换、事务边界
↓ 调用 ↑ 返回		
数据访问层	@Mapper · DemandMapper	执行 SQL、对象与数据行映射
↓ 调用 ↑ 返回		
持久化层	MySQL · demand / acceptance	约束、索引、持久化状态

依赖方向保持单向：Controller 调用 Service，Service 调用 Mapper；禁止 Controller 绕过业务层直接访问数据库。

一个请求如何穿过完整项目？

POST /api/demands

- Tomcat / DispatcherServlet
- DemandController.create(DemandCreateRequest)
- DemandService.create(command)
- DemandMapper.insert(demand)
- MyBatis / JDBC / Connection Pool
- MySQL
- ← generated id ← DemandResponse ← HTTP 201

边界	典型对象	不应承担的职责
Controller	Request/Response DTO	事务、SQL、核心业务判断
Service	Command、领域对象	HTTP 状态码和页面格式
Mapper	PO、查询参数	权限和业务流程
Database	表、约束、索引	用户交互语义

阅读陌生项目时，先从一个 URL 追踪到 Controller、Service、Mapper 和 SQL，再反向追踪响应对象。

统一异常响应

```
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(DemandNotFoundException.class)
    ResponseEntity<ApiError> notFound(DemandNotFoundException e) {
        return ResponseEntity.status(404)
            .body(new ApiError("DEMAND_NOT_FOUND", e.getMessage()));
    }
}
```

可预期错误

转换为稳定的状态码、错误码和用户可理解信息。

未预期错误

记录追踪信息，对外隐藏堆栈、SQL 和内部路径。

PART 3

数据库与 MyBatis

从需求到表：三层数据建模



类图不能机械复制成表；订单、评价、接单等“关系”经常需要成为有状态、有时间的独立实体。

SQL 与约束共同守住数据

```
SELECT id, title FROM demand
WHERE status = 'OPEN' ORDER BY created_at DESC LIMIT 20;
```

```
INSERT INTO demand(owner_id, title, status)
VALUES (?, ?, 'OPEN');
```

```
UPDATE demand SET status = 'CANCELLED'
WHERE id = ? AND owner_id = ? AND status = 'OPEN';
```

能力	典型手段
数据定义 DDL	CREATE / ALTER TABLE、索引、约束
数据操作 DML	SELECT / INSERT / UPDATE / DELETE
唯一性	UNIQUE(demand_id) 防止重复订单
引用完整性	FOREIGN KEY 防止引用不存在的用户或需求
查询性能	针对筛选与排序设计联合索引，并用执行计划验证

应用校验提供友好错误，数据库约束提供最终一致性防线，两者不能互相替代。

从对象到关系表

```
CREATE TABLE demand (  
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,  
  owner_id    BIGINT NOT NULL,  
  title       VARCHAR(80) NOT NULL,  
  category    VARCHAR(30) NOT NULL,  
  location    VARCHAR(120) NOT NULL,  
  status      VARCHAR(20) NOT NULL,  
  provider_id BIGINT NULL,  
  version     INT NOT NULL DEFAULT 0,  
  created_at  TIMESTAMP NOT NULL  
);
```

设计表时先明确：主键、外键、空值、唯一性、状态字段、时间字段和并发控制字段。

JDBC 与 MyBatis：保留原理，减少样板代码

JDBC 手工处理	MyBatis 代为处理
获取与释放连接	连接和会话管理
创建 PreparedStatement	参数绑定
遍历 ResultSet	结果映射
转换 SQL 异常	异常体系集成

```
@Mapper
public interface DemandMapper {
    Demand findById(Long id);
    int insert(Demand demand);
}
```

MyBatis 减少机械代码，但 SQL 的正确性、性能和事务边界仍由开发者负责。

DAO 是职责，Mapper/Repository 是实现方式

Service → Data Access Boundary → Database

└─ MyBatis Mapper

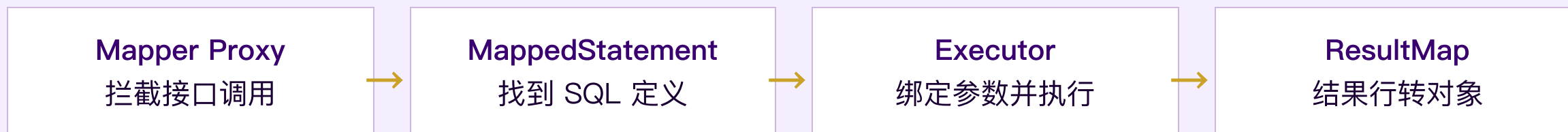
└─ JPA Repository

概念	关注点
DAO / Repository 边界	向业务层提供面向领域的数据库访问接口
MyBatis Mapper	将接口方法映射到显式 SQL 和结果对象
JPA Repository	通过实体和持久化上下文完成数据库访问

```
public interface DemandStore {  
    Optional<Demand> findById(Long id);  
    void save(Demand demand);  
}
```

业务复杂或需要替换持久化技术时，可定义领域侧接口，再由 Mapper/JPA 适配器实现；简单 CRUD 项目不必为了形式重复包装每个方法。

MyBatis 调用时发生了什么？



Spring 管理 `SqlSessionFactory` → 为 Mapper 创建代理
每次调用使用受管理的 `SqlSession` → JDBC `PreparedStatement` → MySQL

- `SqlSessionFactory` 可共享；`SqlSession` 不是线程安全的；
- 在 Spring 项目中不要手工长期保存 `SqlSession`；
- Mapper 接口方法与 XML 的 `namespace + id` 必须对应。

MyBatis 数据源与映射配置

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/campushub
    username: app
    password: ${DB_PASSWORD}
  mybatis:
    mapper-locations: classpath:mapper/*.xml
    type-aliases-package: edu.nju.campushub
    configuration:
      map-underscore-to-camel-case: true
```

```
<mapper namespace="edu.nju.campushub.demand.DemandMapper">
  <select id="findById" resultMap="DemandResult"> ... </select>
</mapper>
```

密码通过环境变量或密钥服务注入，不能提交到 Git；配置加载失败应让应用启动失败，而不是静默连接错误数据库。

MyBatis：注解映射还是 XML 映射？

注解：适合简单 SQL

```
@Insert("""
INSERT INTO demand(owner_id,title,status)
VALUES("#{ownerId}","#{title}","#{status}")
""")
@Options(useGeneratedKeys=true,
         keyProperty="id")
int insert(Demand demand);
```

XML：适合复杂 SQL

```
<select id="search"
        resultMap="DemandResult">
  SELECT ... FROM demand
  <where>
    <if test="status != null">
      status = #{status}
    </if>
  </where>
</select>
```

- 注解靠近接口，便于阅读简单 CRUD；XML 更适合动态 SQL、复杂映射和长查询；
- 一个 Mapper 可以混用，但同一方法只能有一个 SQL 定义来源；
- `namespace` 对应接口全限定名，`id` 对应方法名。

MyBatis CRUD：返回值表达什么？

```
@Mapper
public interface DemandMapper {
    @Select("SELECT id,title,status FROM demand WHERE id=#{id}")
    Demand findById(Long id);

    int insert(Demand demand); // 受影响行数；生成主键可回填到 id
    int updateStatus(@Param("id") Long id,
                    @Param("from") String from,
                    @Param("to") String to);
    int deleteById(Long id);
}
```

```
<update id="updateStatus">
    UPDATE demand SET status = #{to}
    WHERE id = #{id} AND status = #{from}
</update>
```

- 写操作返回值通常是受影响行数，不是更新后的实体；
- 结果为 `0` 可能表示资源不存在或状态冲突，Service 必须定义业务语义；
- 创建后需要主键时，配置数据库生成键或使用应用生成的 ID。

MyBatis 查询与结果映射

```
<select id="findById" resultMap="DemandResult">
  SELECT id, owner_id, title, category, location, status,
         provider_id, version, created_at
  FROM demand
  WHERE id = #{id}
</select>

<resultMap id="DemandResult" type="Demand">
  <id column="id" property="id"/>
  <result column="owner_id" property="ownerId"/>
  <result column="provider_id" property="providerId"/>
</resultMap>
```

显式列名和映射比 `SELECT *` 更稳定，也便于审查数据边界。

动态 SQL：组合条件而不是拼字符串

```
<select id="search" resultMap="DemandResult">
  SELECT id, owner_id, title, category, status, created_at
  FROM demand
  <where>
    <if test="category != null">AND category = #{category}</if>
    <if test="status != null">AND status = #{status}</if>
    <if test="keyword != null and keyword != ''">
      AND title LIKE CONCAT('%', #{keyword}, '%')
    </if>
  </where>
  ORDER BY created_at DESC
</select>
```

`<where>` 会自动处理首个 `AND`；`<if>`、`<choose>`、`<foreach>` 用于安全组合 SQL 结构。参数值仍必须使用 `#{...}`。

动态条件越多，组合测试越重要：无条件、单条件、多条件、空字符串和分页边界都要验证。

SQL 注入：参数值不能拼接

<!-- 安全：生成预编译参数占位符 -->

```
WHERE category = #{category}
```

<!-- 危险：把用户输入直接拼进 SQL -->

```
WHERE category = '${category}'
```

`#{...}` 用于参数值；`${...}` 是文本替换，不能接收未经白名单限制的用户输入。

对于排序字段等无法参数化的位置，使用枚举白名单映射，不能直接使用请求字符串。

事务：一组操作要么全成，要么全败

接单并非一次简单 UPDATE

1. 检查需求是否存在且为 `OPEN`；
2. 检查接单人不是需求发布者；
3. 将需求改为 `ACCEPTED`；
4. 创建订单；
5. 写入双方通知。

```
@Transactional
public OrderResponse acceptDemand(Long demandId, Long providerId) {
    // 多次数据库写入共享一个事务
}
```

事务保证原子性，但并发竞争还需要条件更新、版本号或数据库锁。

并发接单：两个人同时点击怎么办？

```
UPDATE demand
SET status = 'ACCEPTED', provider_id = #{providerId}, version = version + 1
WHERE id = #{id}
  AND status = 'OPEN'
  AND version = #{version};
```

```
int changed = demandMapper.accept(id, providerId, version);
if (changed == 0) throw new DemandConflictException();
```

结果为 0 表示状态已经变化，API 应返回 409 Conflict，而不是覆盖另一位同学的接单结果。

JPA、Hibernate 与 Spring Data JPA

JPA

Java 持久化**规范**：定义实体、关系、查询和生命周期 API。

Hibernate

最常用的 JPA **实现**：负责 ORM、SQL 生成和脏检查。

Spring Data JPA

基于 JPA 的**仓储抽象**：减少常规 CRUD 代码。

业务代码 → JpaRepository → JPA 接口 → Hibernate → JDBC → 数据库

JPA 不是数据库，也不是 Hibernate；它是一套规范。Spring Data JPA 进一步简化了仓储层。

引入 JPA：依赖与配置

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/campusdb
    username: app
    password: ${DB_PASSWORD}
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate
```

配置	课程项目建议
ddl-auto	使用 <code>validate</code> ；表结构交给 Flyway/Liquibase 迁移
open-in-view	关闭，避免在 Controller 隐式查询数据库
SQL 日志	开发环境开启并观察实际查询，生产环境谨慎开启

Entity：把对象映射到表

```
@Entity
@Table(name = "demand")
public class Demand {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String title;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private DemandStatus status;

    @Version
    private long version;

    protected Demand() {} // JPA 使用
}
```

`@Entity` 映射表，`@Id` 标识主键，`@Column` 表达列约束，`@Version` 支持乐观锁。

Entity 不是 API DTO

Entity

服务于持久化和领域状态，可能包含关联、版本号、内部字段。

Demand

DTO

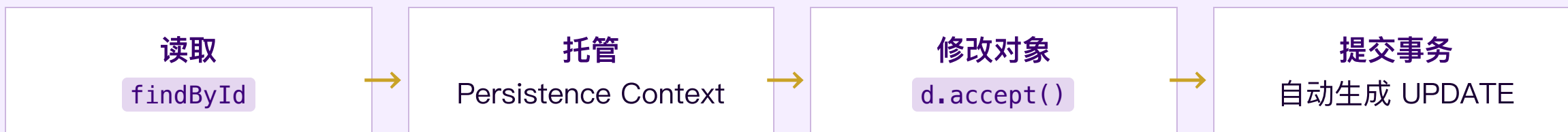
服务于 API 契约，只暴露调用方需要且允许看到的数据。

DemandResponse

```
public record DemandResponse(Long id, String title, String status) {  
    static DemandResponse from(Demand d) {  
        return new DemandResponse(d.getId(), d.getTitle(), d.getStatus().name());  
    }  
}
```

直接返回 Entity 会泄露内部字段，还可能触发关联的延迟加载、循环序列化和不稳定的 API 结构。

持久化上下文与脏检查



```
@Transactional
public void accept(Long id, Long providerId) {
    Demand demand = repository.findById(id)
        .orElseThrow(DemandNotFoundException::new);
    demand.accept(providerId); // 托管实体被修改, 提交时执行脏检查
}
```

- `new`：尚未持久化；`managed`：由当前持久化上下文跟踪；
- `detached`：已脱离上下文；`removed`：等待删除；
- 脏检查依赖事务边界，不能把“自动保存”理解成无条件发生。

Repository: 从 CRUD 到分页

```
public interface DemandRepository extends JpaRepository<Demand, Long> {  
    Page<Demand> findByStatusAndCategory(  
        DemandStatus status, DemandCategory category, Pageable pageable);  
}
```

```
PageRequest page = PageRequest.of(0, 20, Sort.by("createdAt").descending());  
Page<Demand> result = repository.findByStatusAndCategory(  
    DemandStatus.OPEN, DemandCategory.EXPRESS, page);
```

`JpaRepository` 提供 `save`、`findById`、`findAll`、`delete`、分页和排序。方法名派生查询适合**短而稳定**的条件；方法名过长时应改用 JPQL、Specification 或专用查询对象。

JPQL 查询的是实体和属性

```
@Query("""
    select d from Demand d
    where d.status = :status
        and lower(d.title) like lower(concat('%', :keyword, '%'))
    order by d.createdAt desc
    """)
Page<Demand> search(@Param("status") DemandStatus status,
                    @Param("keyword") String keyword,
                    Pageable pageable);
```

查询方式	适合场景
方法名派生	简单等值、排序和少量条件
JPQL	围绕实体关系的自定义查询
Specification	多个可选条件动态组合
Native SQL	数据库特性、复杂报表或性能关键查询

JPQL 中写 `Demand.status` 等实体属性，而不是 `demand.status_code` 等表和列名。

实体关系：明确谁维护外键

```
@Entity
class Demand {
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "owner_id", nullable = false)
    private User owner;
}

@Entity
class User {
    @OneToMany(mappedBy = "owner")
    private List<Demand> demands = new ArrayList<>();
}
```

- `Demand.owner` 持有 `owner_id`，是关系的维护方；`mappedBy` 指向维护方属性；
- `ManyToOne` 默认是 EAGER，项目中通常显式设为 `LAZY`；
- 不要随意使用 `CascadeType.ALL`，尤其不要让删除用户级联删除全部业务记录；
- 双向关系需要在领域方法中同步两端，但 API 仍应返回 DTO。

懒加载与 N+1 查询

1 次: `SELECT * FROM demand WHERE status = 'OPEN'`
N 次: 对每条 Demand 再 `SELECT * FROM user WHERE id = ?`

```
@Query("""
    select d from Demand d
    join fetch d.owner
    where d.status = :status
    """)
List<Demand> findWithOwner(@Param("status") DemandStatus status);
```

症状

列表只有 20 条，日志却出现 21 次甚至更多 SQL。

处理

按用例使用 `fetch join`、`@EntityGraph` 或 DTO 投影，并用 SQL 日志验证。

不要把所有关系改成 EAGER；那会把问题变成不可控的大查询和对象图加载。

JPA 中的事务与并发

```
@Transactional
public void accept(Long demandId, Long providerId) {
    Demand demand = repository.findById(demandId)
        .orElseThrow(DemandNotFoundException::new);
    demand.accept(providerId);
}
```

`@Version` 会把版本号加入 `UPDATE` 条件；两人同时接单时，后提交者得到乐观锁异常，应转换为 `409 Conflict`。

原则	说明
事务放在 Service	一个业务用例形成一个原子边界
避免长事务	不在事务中等待远程调用或用户输入
只读查询	可使用 <code>@Transactional(readOnly = true)</code> 表达意图
批量更新	JPQL update 绕过实体状态，需注意清理持久化上下文

MyBatis 与 JPA 如何选择？

维度	MyBatis	Spring Data JPA
SQL	开发者显式编写	框架生成或 JPQL
建模中心	表、SQL 和结果映射	实体、关系和持久化上下文
复杂查询	SQL 控制直接	JPQL/Specification, 复杂报表可能转原生 SQL
CRUD 速度	映射代码略多	Repository 开发较快
性能风险	SQL 本身和映射错误	隐式查询、N+1、抓取范围和脏检查
适合场景	SQL 密集、报表和精细性能控制	领域对象关系清晰、常规 CRUD 较多

选择依据是团队经验、查询复杂度和领域模型，而不是“哪一个更高级”。课程项目应统一一种主持持久化方案，避免同一业务模块同时使用两套抽象。

PART 4

安全、测试与 AI 协作

后端安全要守住四个边界

认证

调用者是谁？令牌是否有效？

授权

此用户能否操作此资源？

输入

格式、长度、枚举和业务约束是否合法？

输出

是否泄露密码、联系方式或内部错误？

“前端没有显示这个按钮”不是授权。每一个写操作都必须在后端重新检查身份和资源关系。

认证与授权是两件事

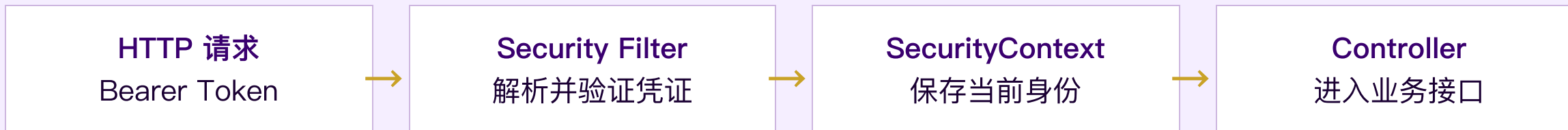
认证 Authentication: 这个请求来自用户 42

授权 Authorization: 用户 42 是否可以取消需求 101

```
Demand demand = requireDemand(id);  
if (!demand.ownerId().equals(currentUserId)) {  
    throw new AccessDeniedException("not owner");  
}
```

- 密码必须使用专用密码哈希算法存储；
- Token 只能证明身份，不能自动证明操作权限；
- 管理员接口与普通用户接口需要不同授权规则。

Spring Security: 请求先经过过滤器链



```
@Bean
SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .build();
}
```

URL 级规则是第一层；“是否拥有需求 101”仍应在 Service 中做资源级授权。

JWT：签名令牌不是加密保险箱

登录：学号 + 密码 → 验证密码哈希 → 签发短期 Access Token

调用：Authorization: Bearer <token> → 验签 → 检查 exp/iss/aud → 建立身份

刷新：Refresh Token → 轮换并签发新的 Access Token

JWT 内容	原则
sub	稳定的用户标识，不使用昵称
exp	必须设置较短有效期
iss / aud	限定签发方和使用方
roles/claims	只放必要授权线索，服务端仍校验资源关系

JWT Payload 默认只是 Base64URL 编码，任何持有者都能读取。不得放密码、联系方式或其他敏感信息。

密码、CORS、CSRF 与依赖安全

风险	后端措施
密码泄露	使用 <code>PasswordEncoder</code> (BCrypt/Argon2), 每个密码独立加盐
CORS	只允许可信前端 Origin、必要方法和请求头
CSRF	Cookie 会话必须防护; 纯 Bearer Token API 需避免把 Token 自动放入跨站请求
XSS	输出编码、限制富文本; Token 存储策略需评估被脚本读取的风险
依赖漏洞	锁定并更新依赖, 启用 Dependabot/OWASP Dependency-Check 等扫描
密钥泄露	使用环境变量或密钥服务, 定期轮换, 不写入仓库和日志

安全不是加入一个 Starter 就完成, 而是身份、授权、输入、输出、配置和运维共同形成的边界。

后端测试按风险分层

测试	重点验证
Service 单元测试	状态转换、权限、边界和异常
Controller 测试	参数绑定、状态码、JSON 契约
Mapper 集成测试	SQL、映射、约束和事务
端到端测试	前后端关键流程真正连通

```
@Test
void ownerCannotAcceptOwnDemand() {
    assertThrows(SelfAcceptException.class,
        () -> service.accept(DEMAND_ID, OWNER_ID));
}
```

案例测试：接单状态机

场景	预期结果
未登录用户接单	401 Unauthorized
发布者接自己的单	403 Forbidden
接一个不存在的需求	404 Not Found
接一个已被接的需求	409 Conflict
合法接单	201 Created，状态变为 ACCEPTED
两人并发接单	仅一人成功，另一人得到 409

正常流程只是一个测试；工程风险集中在失败、越权和并发路径。

不要只说“帮我写一个后端接口”

可执行任务

在 Spring Boot + MyBatis 项目中实现 POST /api/demands/{id}/acceptance。

只允许修改 demand 模块。输入为当前用户 id 和需求 id。

规则：发布者不能接单；仅 OPEN 状态可接；并发时只能一人成功。

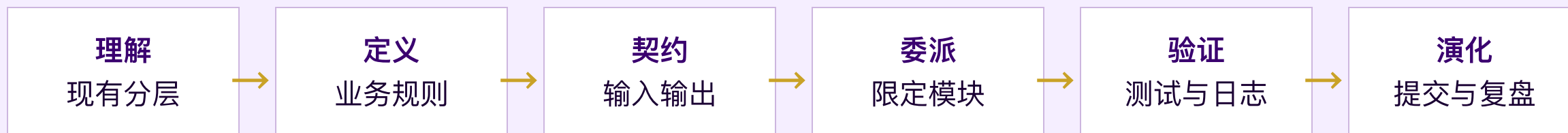
成功返回 201；不存在返回 404；冲突返回 409；越权返回 403。

使用条件 UPDATE 和事务，不使用 \${} 拼接参数。

补充 Service 单元测试、Controller 测试和 Mapper 集成测试。

规格中必须包含：边界、状态、失败语义、并发策略和验证方式。

后端小任务的 AI 协作六步法



AI 不能替你决定的内容

- 谁拥有资源、哪些状态允许转换；
- 事务和并发策略；
- 哪些数据可以对外暴露；
- 验收测试是否覆盖真实风险。

审查 AI 生成的后端代码

检查项	典型问题
API 契约	状态码、字段名与前端约定不一致
分层	Controller 直接操作 Mapper
事务	多次写入未放在同一事务中
并发	先查后改，导致重复接单
安全	缺少资源级授权或拼接 SQL
错误	把堆栈和数据库信息返回给用户
测试	只有正常流程，没有失败和并发场景

能运行只是起点；正确性取决于契约、状态、事务、安全和验证。

小练习：实现需求详情 API

实现 `GET /api/demands/{id}`：

1. 定义 `DemandResponse`，不暴露联系方式等敏感字段；
2. Controller 只处理 HTTP 边界；
3. Service 处理“不存在”和可见性规则；
4. Mapper 使用显式列名查询；
5. 返回正确的 `200 / 403 / 404`；
6. 编写三层测试并记录 AI 生成代码的人工修正。

提交： API Spec、代码、测试结果、一次失败案例和 Git diff。

本讲知识地图

HTTP / REST: URL、JSON、请求、响应、状态码、API 契约

Spring Boot: 项目结构、路由、DTO 校验、Controller、异常响应

业务层: Service、状态规则、事务与并发

数据层: 关系建模、SQL、MyBatis、JPA、参数绑定、结果映射

质量保障: Spring Security、JWT、分层测试、AI 代码审查

最重要的五个结论

1. 后端是业务规则和数据正确性的最终防线；
2. API 契约先于代码，输入、输出和失败语义必须明确；
3. Controller、Service、Mapper 各有职责，分层为了隔离变化和便于测试；
4. 事务、并发和授权不能靠前端补救；
5. AI 能快速生成代码，但必须用测试、日志和数据库结果验证。

课后任务

必做

完成 CampusHub “接单”接口并提交：

01 API 契约及错误码表

03 正常、越权、不存在、冲突和并发测试

05 200 字反思：AI 最容易遗漏哪类后端风险？

02 Controller、Service、Mapper 实现

04 AI 协作记录：任务 Spec、关键输出、人工修改

选做

- 增加幂等键，避免重复提交；
- 增加审计日志和接口耗时监控。

Java Web 后端开发基础

从“接口能返回”走向“规则可信、数据一致、结果可验证”

下一讲：软件工程基础